FINAL YEAR CAPSTONE PROJECT DEFENSE

APIFORGE

Advanced API Load-Testing & Performance-Analysis Platform

Comprehensive Codebase & Viva Voce Defense Guide: An exhaustive, file-by-file, line-by-line examination of the architecture, technical choices, security fortifications, design patterns, and examiner questions.

Project Title: APIForge Monorepo Platform

Technology Stack: Node.js, Express, React 18, Vite, MongoDB, k6

Architecture: Modular Monolith, Dual HTTP Servers, SPA

Security Protections: Multi-layer SSRF, JWT, Bcrypt, Helmet, CORS, Zod

Verification Status: 131/131 Tests Passing (Unit & Integration)

Document Purpose: Viva Defense, Code Walkthrough & Oral Examination

Academic Year 2026

Confidential & Proprietary Project Documentation

Table of Contents

Section	Topic & Core Modules	Focus Area
Section 1	**Project Overview & High-Level System Architecture**	System Design & Tech Justification
Section 2	**Technology Stack: Deep Justifications & Alternatives**	Why React+Vite, Express, Mongo, k6
Section 3	**Backend Core & Server Infrastructure** (server.js, app.js, demoApp.js)	Dual Servers, Graceful Shutdown, Pipelines
Section 4	**Configuration & Database Layer** (env.js, database.js)	Zod Validation, Connection Pooling, Events
Section 5	**Middleware Pipeline & Security Architecture**	Error Handling, Auth, Zod Validation, Rate Limits
Section 6	**SSRF Defense & Security Utilities** (ssrfFilter.js, response.js, AppError.js)	DNS Rebinding, Cloud Metadata, OOP Errors
Section 7	**Authentication & Authorization Module** (user.model.js, jwt.service.js, auth.service.js)	Bcrypt Salts, JWT Claims, select:false
Section 8	**Target API Management Module** (target.model.js, target.service.js)	SSRF Preflights, AbortController Health Probes
Section 9	**Isolated Built-in Demo Sandbox** (demoApp.js, demo.controller.js)	Port 5001, Latency, Error & Rate Injection
Section 10	**Test Configuration & Finite State Machine Engine** (testRun.model.js, test.service.js)	FSM Transitions, Invariants, Concurrency Limits
Section 11	**Frontend Architecture & React 18 Implementation**	Axios Interceptors, Context, Polling Engine
Section 12	**Load Testing & Performance Metrics Formulations**	VUs, RPS, P90/P95 Percentiles vs Averages
Section 13	**Testing Strategy & Verification Framework**	Vitest, Supertest, React Testing Library
Section 14	**Top 50 Viva Examiner Questions & High-Scoring Answers**	Oral Examination Rapid-Fire Q&A

Section 1: Project Overview & High-Level Architecture

1.1 What is APIForge?

APIForge is a high-performance, developer-centric API load-testing and performance-analysis platform. It allows software engineers and DevOps teams to define target endpoints, configure virtual user (VU) traffic profiles, execute real load tests, monitor execution in real time, extract normalized latency distributions (Average, Median, P90, P95, Max), and compare benchmark runs.

1.2 The Problem It Solves

Existing commercial load-testing platforms (such as Postman Enterprise, BlazeMeter, or Datadog Synthetic Monitoring) are either expensive, closed-source, or introduce heavy cloud latency and data-residency concerns. On the other hand, raw CLI tools like `k6` or `wrk` lack persistent test history, real-time visual dashboards, tenant isolation, team collaboration, and automated SSRF security defenses. APIForge bridges this gap with an intuitive, self-hosted full-stack platform.

> **Architecture Blueprint: Decoupled Dual-HTTP Server Topology**
>
> APIForge runs two independent HTTP servers within a single Node.js runtime process:
>
> - **Port 5000 (Main Management API):**
>
> Handles authentication, target management, test scheduling, result persistence, and metrics aggregation. Protected by strict JWT authentication, CORS, and rate-limiting.
> - **Port 5001 (Isolated Demo Sandbox API):**
>
> An unauthenticated, zero-interference benchmarking target offering instant ping, configurable latency injection (0–5000ms), probabilistic HTTP error injection (400–504), and sliding-window rate-limiting.

> **Top Viva Question: Why run a separate Demo Server on Port 5001 instead of mounting it on Port 5000?**
>
> *Answer:* If the load testing engine fired hundreds of concurrent Virtual Users (VUs) directly at Port 5000, it would saturate the Node.js event loop of our management server, trigger our own rate limiters, lock our MongoDB connection pool, and starve authenticated users from using the dashboard. By running the Demo Sandbox on Port 5001 with independent middleware pipelines, high-throughput benchmarking never compromises dashboard responsiveness or data integrity.

Section 2: Technology Stack Justifications

Examiners routinely ask "*Why did you choose X over Y?*". Use the following structured technical arguments:

Technology	Role in APIForge	Why Chosen (Technical Advantages)	Rejected Alternative & Rationale
Node.js (ES Modules)	Backend Runtime Engine	Non-blocking, single-threaded asynchronous event loop ideal for orchestrating I/O-heavy operations (spawning child processes, streaming network probes, querying MongoDB). Modern ES Modules (`import/export`) provide clean module resolution.	**Python/Django:** Multi-threaded/WSGI overhead, slower asynchronous I/O primitives for real-time progress polling compared to Node's libuv event loop.
Express.js	Web Framework	Minimalist, unopinionated, robust middleware architecture allowing surgical control over request pipelines, custom error handling, and dual server instantiation.	**NestJS:** Unnecessary enterprise boilerplate and heavy TypeScript decorator overhead for this micro-service style platform.
MongoDB + Mongoose	Database & ODM Layer	Dynamic JSON document storage perfectly matches variable test configurations, custom header maps, and nested metric histograms. Schema validation and pre/post hooks provide strict data modeling.	**PostgreSQL:** Relational schemas require rigid migrations for varying header structures, polymorphic error payloads, and arbitrary metric timeseries maps.
React 18 + Vite	Frontend SPA & Tooling	Instant Hot Module Replacement (HMR) powered by native ES modules (esbuild/Rollup), concurrent rendering in React 18, zero build bloat, and fast component updates.	**Create React App (CRA):** Deprecated, slow Webpack bundling, bloated configuration. **Next.js:** Unnecessary SSR overhead for a private, authenticated dashboard SPA.
Tailwind CSS	Styling & Design System	Utility-first CSS with zero runtime penalty, consistent design tokens for our custom dark developer theme, responsive breakpoints, and tiny production bundle footprint via PurgeCSS.	**Bootstrap:** Generic appearance, heavy unused CSS rules. **Material UI:** Heavy runtime JavaScript CSS-in-JS performance penalty.
Zod	Authoritative Validation	TypeScript-first, runtime schema declaration and parsing. Performs input coercion, trimming, sanitization, and structured error reporting simultaneously.	**Joi / Express-Validator:** Cumbersome syntax, does not provide automatic type casting or strip unknown fields as cleanly as Zod.
k6 (Grafana)	Load Testing Engine	Written in Go, k6 provides exceptional memory efficiency (goroutines vs heavy OS threads), supports high Virtual User (VU) concurrency without Java JVM bloat, and exports structured JSON metrics.	**Apache JMeter:** Java JVM memory hog, heavy XML configuration, difficult to orchestrate programmatically. **Artillery:** Node-based, suffers event loop starvation under high VUs.
Vitest	Testing Framework	Shares the exact same configuration, plugins, and transform pipeline as Vite. Native ESM support, built-in watch mode, multi-threading via worker	**Jest:** Painful configuration for native ES Modules, slow Babel transforms, significantly slower test execution across monorepos.

Technology	Role in APIForge	Why Chosen (Technical Advantages)	Rejected Alternative & Rationale
		threads, compatible with Jest assertions.	

Section 3: Backend Core & Server Infrastructure

3.1 File: `server/src/server.js` (Process Entry & Lifecycle)

This file is the executable entry point. It imports `app.js` and `demoApp.js`, connects to MongoDB, starts HTTP listeners, and configures OS signal handlers for graceful shutdown.

> **Crucial Examiner Question: Why separate `server.js` from `app.js`?**
>
> *Answer:* If you call `app.listen()` inside `app.js`, then whenever an integration test runs (using Supertest), the test framework will attempt to bind to Port 5000. If the server is already running, tests crash with `EADDRINUSE` (Port already in use). Separating `app.js` (which exports the un-listened Express app) from `server.js` (which binds to the port) allows Supertest to test the HTTP pipeline in-memory on ephemeral random ports without port conflicts!

Lines	Code Snippet	Why It's Written (Technical Purpose)	Consequence if Omitted
`10-12`	`connectDatabase().catch(err => ...);`	Attempts MongoDB connection asynchronously without blocking HTTP server boot. If DB is down, server boots in offline mode so health checks still respond.	Server would crash on startup if MongoDB had a 1-second network blip.
`14-20`	`const server = app.listen(5000);` `const demoServer = demoApp.listen(5001);`	Spawns two separate HTTP listener instances on dedicated ports within the same Node process.	Cannot isolate load testing benchmarks from user management traffic.

`23-40`	`const handleShutdown = async (signal) => {`
	`  demoServer.close();`
	`  server.close(async () => {`
	`  await disconnectDatabase();`
	`  });`
	`};`
		Graceful Shutdown: Stops accepting new incoming HTTP connections, finishes existing requests in flight, cleanly closes MongoDB socket connections, and releases ports.	Abrupt process exit leaves orphaned TCP sockets, corrupts in-flight database writes, and causes 502 Bad Gateway on proxies.

`43-46`	`setTimeout(() => {`
	`  process.exit(1);`
	`}, 10000).unref();`
		Safety Net Timer: If pending connections hang indefinitely during shutdown, forces exit after 10 seconds. `.unref()` ensures this timer does NOT keep the event loop alive if everything closes cleanly earlier.	Without `.unref()`, Node.js would be forced to wait the full 10 seconds on every normal shutdown!

`49-50`	`process.on('SIGINT', ...);` `process.on('SIGTERM', ...);`	Listens for POSIX termination signals: `SIGINT` (triggered by Ctrl+C in terminal) and `SIGTERM` (triggered by Kubernetes, Docker, or systemd).	Container orchestrators would kill the process abruptly via `SIGKILL` after timeout.

3.2 File: `server/src/app.js` (Express Middleware Pipeline)

This file instantiates Express, attaches security middlewares in strict chronological order, defines the health check, mounts API sub-routers, and attaches error handling.

Security Alert: Why is Middleware Order in Express Critical?

In Express, middlewares execute in the exact order they are registered via `app.use()`. If `errorHandler` were registered at the top, it would never catch errors thrown by route handlers below it. If `express.json()` were placed after route definitions, `req.body` would be undefined inside controllers!

Lines	Code Snippet	Why It's Written (Technical Purpose)	Viva Trap / Examiner Q
`21`	`app.use(helmet());`	Sets 15+ secure HTTP response headers (e.g. `X-Content-Type-Options: nosniff`, `X-Frame-Options: SAMEORIGIN`, `Strict-Transport-Security`, `Content-Security-Policy`) to protect against XSS, clickjacking, and MIME sniffing.	"Name 3 HTTP headers Helmet adds and what attack each prevents."
`24-29`	`app.use(cors({ origin: env.CLIENT_URL, credentials: true }));`	Restricts Cross-Origin requests specifically to the authorized frontend domain (`http://localhost:5173`) while allowing credentials (cookies, authorization headers).	"What happens if origin is set to '*' with credentials: true?" (Browsers reject this combination by CORS specification for security).
`32-34`	`if (env.NODE_ENV !== 'test') { app.use(morgan('dev')); }`	Logs HTTP requests with status codes and response times to the console during development, but silences logs during automated tests.	"Why disable logging in test mode?" (Prevents log pollution in test runners like Vitest).
`37`	`app.use('/api', apiLimiter);`	Applies global IP rate-limiting to all `/api` endpoints before request parsing or database querying.	"Why place rate limiter before route handlers?" (Drops abusive bots before wasting CPU/RAM on parsing JSON or database queries).
`40-41`	`app.use(express.json({ limit: '1mb' }));`	Parses incoming JSON request payloads with a strict 1 megabyte ceiling.	"Why enforce a 1MB limit?" (Protects against Denial-of-Service attacks where an attacker streams a 500MB JSON payload to crash Node's memory).
`44-57`	`app.get('/api/health', ...)`	Exposes server uptime, version, and live MongoDB connection state without requiring authentication. Used by Kubernetes/Docker liveness probes.	"What HTTP status should health check return if DB is disconnected?" (Can still return 200 with `database: 'disconnected'` so load balancers don't instantly terminate the pod during brief reconnections).
`74-77`	`app.use(notFound); app.use(errorHandler);`	`notFound` catches all unmatched URLs and instantiates a `NotFoundError`; `errorHandler` is the final 4-argument handler that formats all errors uniformly.	"How does Express recognize error middleware?" (By having exactly 4 parameters: `err, req, res, next`).

Section 4: Configuration & Database Layer

4.1 File: `server/src/config/env.js` (Startup Environment Validation)

This file loads the `.env` file and validates every configuration variable against a strict Zod schema before the server boots.

> **Security Invariant: The 32-Character JWT Secret Rule**
>
> In line 17-19 of `env.js`, Zod enforces: `JWT_SECRET: z.string().min(32, { message: 'JWT_SECRET must be at least 32 characters long...' })`. If a developer tries to run the server with a weak secret like `"secret123"`, the application crashes immediately on startup!
>
> *Viva Rationale:* HMAC-SHA256 tokens signed with short, dictionary-guessable secrets can be cracked offline in seconds using tools like `hashcat`. Enforcing 256 bits (32 bytes) of entropy prevents brute-force token forgery.

4.2 File: `server/src/config/database.js` (MongoDB Connection Manager)

Manages the Mongoose ODM connection lifecycle, implements connection pooling, handles auto-reconnect, and isolates test databases.

Lines	Code Snippet	Why It's Written (Technical Purpose)	Viva Questions & Answers
`11`	`const uri = customUri || (env.NODE_ENV === 'test' ? env.MONGODB_TEST_URI : env.MONGODB_URI);`	Dynamic URI resolution: Automatically points to `apiforge_test` in test mode and `apiforge` in dev/prod.	"Why use a separate test database?" (To ensure automated tests never drop or corrupt real development/production data).
`14-16`	`if (mongoose.connection.readyState === 1) { return mongoose; }`	Singleton connection check: If already connected (readyState 1), returns immediately to avoid duplicate socket pools.	"What are the 4 values of readyState?" (0: Disconnected, 1: Connected, 2: Connecting, 3: Disconnecting).
`20`	`serverSelectionTimeoutMS: 5000`	Limits database driver connection timeout to 5 seconds (default in Mongoose is 30s).	"Why reduce timeout to 5s?" (Fails fast during startup if database is down rather than hanging user requests for 30s).
`21`	`autoIndex: env.NODE_ENV !== 'production'`	Disables automatic Mongoose index building in production environments.	"Why disable autoIndex in production?" (Building indexes automatically at runtime creates major performance spikes and collection locks on large databases. In production, indexes must be created via migration scripts).
`56-66`	`mongoose.connection.on('disconnected', ...);` `mongoose.connection.on('reconnected', ...);`	Event-driven connection monitoring: Emits warnings and logs when MongoDB connectivity is interrupted and restored.	"How does Mongoose handle temporary connection drops?" (It automatically queues incoming queries in a 30s buffer and executes them when reconnected).

Section 5: Middleware Pipeline & Security Architecture

5.1 File: `server/src/middleware/errorHandler.js` (Centralized Error Pipeline)

This is the most critical resilience file in the backend. It intercepts all synchronous and asynchronous errors forwarded via `next(err)` and categorizes them.

Top Viva Question: What is the difference between Operational Errors and Programmer Errors?

- **Operational Errors (Trusted):**
  Expected runtime errors that inevitable occur during normal application usage (e.g. invalid input, unauthorized access, resource not found, duplicate email). These inherit from `AppError`, have `isOperational = true`, and return informative HTTP 4xx responses.
- **Programmer Errors (Bugs):**
  Unexpected failures caused by code mistakes (e.g. `TypeError: cannot read property of undefined`, syntax errors, failed database connections). In production, these return a generic `500 Internal Server Error` and conceal stack traces to prevent security leakage.

Lines	Code Snippet	Why It's Written (Technical Purpose)	Consequence if Omitted
`13-15`	`if (res.headersSent) { return _next(err); }`	Checks if HTTP response headers were already sent to the client. If so, delegates to Express default handler.	Attempting to call `res.status().json()` after headers are sent crashes the Node process with `ERR_HTTP_HEADERS_SENT`.
`25-27`	`else if (err instanceof SyntaxError && err.status === 400 && 'body' in err)`	Catches malformed JSON payloads sent by clients in `req.body` and transforms them into a clean `BadRequestError`.	Server would crash or return an unformatted HTML error page when a client sends invalid JSON syntax.
`30-36`	`else if (err.name === 'ValidationError') { ... }`	Intercepts Mongoose database-level schema validation errors and extracts clean field-by-field error messages.	Exposes raw internal Mongoose schema validation objects to the API consumer.
`39-41`	`else if (err.name === 'CastError') { ... }`	Intercepts invalid MongoDB ObjectIDs (e.g. passing `"abc"` instead of a 24-character hex ID in `/api/targets/abc`).	Returns a generic 500 error instead of a helpful `400 Bad Request: Invalid format for parameter`.
`44-47`	`else if (err.code === 11000) { ... }`	Intercepts MongoDB unique index violation errors (e.g. attempting to register an existing email) and formats a `409 Conflict`.	Exposes raw database constraint names and internal collection metadata.
`54-56`	`env.NODE_ENV === 'production' ? 'An unexpected internal error occurred' : err.message`	Information Hiding: In production, hides internal error messages and stack traces from malicious users.	Attackers could use stack traces to discover server file paths, dependency versions, and database schemas.

5.2 File: `server/src/middleware/authenticate.js` (JWT Route Guard)

Extracts Bearer tokens, verifies cryptographic signatures, and checks user account liveness before permitting access to protected routes.

Lines	Code Snippet	Why It's Written (Technical Purpose)	Viva Trap / Examiner Q
`7-11`	`if (!authHeader || !authHeader.startsWith('Bearer '))`	Enforces standard RFC 6750 OAuth 2.0 Bearer token format in the HTTP `Authorization` header.	"Why use the Authorization header instead of putting tokens in request body or URL query?" (URL queries are stored in server access logs and browser history; request bodies don't exist in GET requests).
`18`	`const decoded = verifyToken(token);`	Verifies token signature using HMAC-SHA256 and checks expiration time (`exp` claim). Throws if tampered or expired.	"What happens if the token was signed with a different secret?" (`jwt.verify` throws `JsonWebTokenError: invalid signature`).
`21-24`	`const user = await User.findById(decoded.id); if (!user) throw new UnauthorizedError(...);`	**Account Liveness Check:** Queries the database to verify the user record still exists and was not deleted after the token was issued.	"Why query the database if JWT is stateless?" (If a user is banned or deleted, a purely stateless JWT would remain valid until its expiration date! Checking existence guarantees immediate revocation).
`26`	`req.user = user;`	Attaches the sanitized user document to the Express `req` object so all downstream controllers know who made the request.	Enables tenant isolation: controllers can query `Target.find({ owner: req.user._id })` .

5.3 File: `server/src/middleware/validateRequest.js` (Zod Request Interceptor)

A higher-order middleware factory that declaratively validates `req.body`, `req.query`, and `req.params` against Zod schemas.

Crucial Code Nuance: Why reassign `req.body = await schemas.body.parseAsync(req.body);`?

Answer: Zod is not just a validator; it is a **data transformer**. Zod schemas in our project use `.trim()`, `.toLowerCase()`, `z.coerce.number()`, and default values. By reassigning `req.body`, `req.query`, and `req.params` to the parsed result:

- Strings are automatically trimmed of whitespace.
- URL query parameters (which arrive as strings like `"15"`) are automatically coerced into real JavaScript numbers.
- Unexpected malicious properties injected by attackers are automatically stripped away!

Section 6: SSRF Defense & Security Architecture

6.1 File: `server/src/shared/utils/ssrfFilter.js` (Multi-Tier SSRF Engine)

Because APIForge allows users to input target URLs to load-test, it is inherently exposed to **Server-Side Request Forgery (SSRF)**. If left unprotected, an attacker could instruct APIForge to attack the host server's loopback, internal company microservices, or cloud instance metadata.

What is SSRF and Why is APIForge Vulnerable Without This Filter?

In an SSRF attack, the attacker submits a target like `http://169.254.169.254/latest/meta-data/iam/security-credentials/`. When the server executes the test or health check, the cloud metadata service sees an internal request originating from the server itself and hands over the AWS/GCP administrative IAM access keys!

Lines	Code Snippet	Why It's Written (Technical Purpose)	Vulnerability Blocked
`7-12`	`const BLOCKED_HOSTNAMES = new Set(['localhost', 'metadata.google.internal', ...]);`	Fast-lookup blacklist blocking known loopback and cloud metadata internal hostnames.	Blocks GCP internal metadata endpoint and basic localhost probes.
`24-41`	`if (net.isIPv6(ip)) { ... }`	Inspects IPv6 addresses for loopback (`::1`), link-local (`fe80::`), unique local (`fc00::` , `fd00::`), and IPv4-mapped loopback addresses (`::ffff:127.0.0.1`).	Bypassing IPv4 filters by submitting IPv6 representations of localhost.

`51-66`	`if (p0 === 127) return true;`
	`if (p0 === 10) return true;`
	`if (p0 === 172 && p1 >= 16 && p1 <= 31) return true;`
	`if (p0 === 192 && p1 === 168) return true;`
	`if (p0 === 169 && p1 === 254) return true;`
		Enforces RFC 1918 Private Network filtering:
		• `127.0.0.0/8` (Localhost)
		• `10.0.0.0/8` (Class A LAN)
		• `172.16.0.0/12` (Class B LAN)
		• `192.168.0.0/16` (Class C LAN)
		• `169.254.0.0/16` (Link-Local & Cloud Metadata)
			Blocks scanning of internal company intranets, routers, and AWS/Azure metadata services.

`115-119`	`if (parsed.protocol !== 'http:' && parsed.protocol !== 'https:')`	Restricts protocol schemes strictly to HTTP and HTTPS.	Blocks protocol smuggling attacks using `file:///etc/passwd` , `gopher://` , or `ftp://` .
`150-166`	`const { address } = await dns.lookup(hostname);` `if (isPrivateIp(address)) throw new ValidationError(...);`	**DNS Resolution & Rebinding Defense:** Performs asynchronous DNS lookup to find the physical IP behind a domain name before allowing it.	**DNS Rebinding Attack:** An attacker registers `attacker.com` pointing to `127.0.0.1` . String checks think it's a valid public domain, but DNS lookup catches the private IP!
`121-129`	`if (options.allowDemo && isDemoTarget(parsed)) { return { ... isDemo: true }; }`	**Sanctioned Whitelist Exception:** Explicitly permits targeting the local Demo Sandbox API ONLY if the port matches `env.DEMO_PORT` .	Allows safe local benchmarking without blowing open arbitrary localhost ports.

Section 7: Authentication & Authorization Module

7.1 File: `server/src/modules/auth/user.model.js` (Mongoose User Schema)

Lines	Code Snippet	Why It's Written (Technical Purpose)	Viva Questions & Answers
`16-20`	`email: { type: String, unique: true, lowercase: true, trim: true, index: true }`	Normalizes emails to lowercase to prevent duplicate account creation (e.g. `User@domain.com` vs `user@domain.com`). Creates unique B-tree index in MongoDB.	"What happens at the database level when unique: true is set?" (MongoDB creates an index constraint. If a duplicate is inserted, it throws error code 11000).
`22-26`	`passwordHash: { type: String, select: false }`	**Crucial Security Control:** `select: false` prevents Mongoose from including the password hash in standard `User.find()` or `User.findById()` queries.	"How do you query the password hash when validating login credentials?" (Use `User.findOne({ email }).select('+passwordHash')`).
`30-36`	`toJSON: { transform: (doc, ret) => { delete ret.passwordHash; delete ret.__v; return ret; } }`	**Defense-in-Depth Serialization Hook:** Whenever a user document is serialized to JSON (e.g. `res.json(user)`), automatically deletes `passwordHash` and Mongoose internal version key `__v`.	Guarantees password hashes never leak to the client even if a developer forgets `select: false`.
`59-62`	`userSchema.statics.hashPassword = async function(password) { const salt = await bcrypt.genSalt(10); return bcrypt.hash(password, salt); };`	Model static method hashing passwords with Bcrypt using 10 salt rounds (~100ms CPU cost).	"Why 10 rounds of salt?" (Work factor of 10 provides 2^10 = 1024 hashing rounds. It is computationally expensive for brute-force attacks while maintaining fast user login).

Section 8: Target Management Module

8.1 File: `server/src/modules/targets/target.service.js`

Implements multi-tenant CRUD operations and health probe executions.

> **Crucial Architecture Rule: Multi-Tenant Ownership Isolation**
> Notice lines 12, 49, 66, and 92: `Target.findOne({ _id: targetId, owner: userId })`. Every database query strictly filters by both `_id` and `owner: userId`. Even if User A guesses the MongoID of a Target owned by User B, the query returns `null`, preventing cross-tenant data leakage or unauthorized testing!

Lines	Code Snippet	Why It's Written (Technical Purpose)	Viva Questions & Answers
`28-29`	`const { normalizedUrl } = await validateTargetUrl(input.baseUrl);`	Performs SSRF pre-flight validation before creating or updating any target.	"What does URL normalization do?" (Strips trailing slashes, path queries, and normalizes port to prevent duplicate targets).
`130-137`	`const controller = new globalThis.AbortController(); const timeoutId = setTimeout(() => controller.abort(), 5000); const response = await fetch(target.baseUrl, { signal: controller.signal });`	**Resilient Health Probe:** Uses native `fetch` with an `AbortController` to enforce an absolute 5-second timeout on target health checks.	"What happens if the target server hangs or drops packets?" (The timeout fires `controller.abort()`, throwing an `AbortError`, marking the target 'offline' without hanging our server).
`158-165`	`target.lastHealthCheck = { status, statusCode, latencyMs, checkedAt: new Date() }; await target.save();`	Persists probe results directly onto the target document so dashboard users can view historical health without re-probing.	Caches health status to avoid unnecessary network traffic on every page load.

Section 9: Isolated Built-in Demo Sandbox

9.1 File: `server/src/modules/demo/demo.controller.js`

Provides deterministic endpoints for benchmarking without third-party internet dependencies:

- `GET /api/demo/ping`: Instant 200 OK baseline for measuring raw network & framework overhead.
- `GET /api/demo/delayed?ms=250`: Clamped server delay using `setTimeout` to evaluate latency metrics and threshold breaches.
- `GET /api/demo/error?status=500&rate=0.5`: Probabilistic fault injection using `Math.random() < rate` to test error handling.
- `GET /api/demo/rate-limited?limit=10&window=10`: Sliding-window rate-limiting returning HTTP 429 and standard `Retry-After` headers.

Section 10: Test Configuration & Finite State Machine Engine

10.1 File: `server/src/modules/tests/testRun.model.js` (Authoritative Lifecycle State Machine)

Load tests transition through a strict, irreversible state machine: `QUEUED → RUNNING → [COMPLETED | FAILED | CANCELLED]`.

> **Top Viva Question: Why implement a Finite State Machine with Terminal States?**
> *Answer:* In distributed and asynchronous systems, race conditions can occur. For instance, if a user clicks *Cancel* while a worker process is finishing a test, a late worker callback might attempt to mark the test as `COMPLETED`. In our model (lines 20-28): `COMPLETED: [], FAILED: [], CANCELLED: []`. Once a test reaches a terminal state, all future transitions are rejected! This guarantees data integrity and prevents zombie worker processes from overwriting cancelled tests.

Lines	Code Snippet	Why It's Written (Technical Purpose)	Consequence if Omitted
`51-54`	`targetSnapshot: { name: String, baseUrl: String }`	**Immutable Target Snapshot:** Captures the target's name and URL at the exact moment the test is created.	If a user later edits or deletes the Target, historical test reports would lose their URL or show inaccurate historical targets!
`162-163`	`testRunSchema.index({ owner: 1, createdAt: -1 }); testRunSchema.index({ status: 1, owner: 1 });`	Compound B-Tree Indexes: The first optimizes paginated test history sorted by date; the second optimizes concurrency queries counting active tests per user.	Without these indexes, every pagination query or concurrency check performs a slow, full collection scan (O(N) vs O(log N)).
`176-249`	`transitionTo(targetStatus, metadata)`	Encapsulates status transitions, validates allowed moves against `VALID_TRANSITIONS`, records `startTime` / `endTime`, and computes `actualDurationMs`.	Scattered, inconsistent status mutations across controllers.
`254-269`	`testRunSchema.pre('save', function (next) { ... })`	**Defense-in-Depth Pre-Save Hook:** Intercepts direct document property mutations (e.g. `testRun.status = 'queued'`) and rejects illegal transitions.	Prevents accidental bypass of the state machine by other developers.
`271-279`	`post('init')` and `post('save')` hooks	Stores `this.$locals.originalStatus = this.status` in document memory upon retrieval from MongoDB.	Enables the `pre('save')` hook to know what the previous state was before the update.

10.2 File: `server/src/modules/tests/test.service.js` (Concurrency & Execution Engine)

> **Resource Protection: Per-User Concurrency Cap**
> In lines 29-39, before creating a test run, the service queries: `await TestRun.countDocuments({ owner: userId, status: { $in: ['queued', 'running'] } })`. If `activeCount >= MAX_CONCURRENT_RUNS_PER_USER (2)`, it throws a `409 ConflictError`.
>
> *Viva Rationale:* Running load tests consumes significant CPU, RAM, and network sockets (spawning k6 virtual users). Without concurrency caps, a single user could queue 50 load tests simultaneously and exhaust the server's thread pool and network interface!

Section 11: Frontend Architecture & React 18 Implementation

11.1 File: `client/src/services/apiClient.js` (Axios Interceptors)

Lines	Code Snippet	Why It's Written (Technical Purpose)	Viva Questions & Answers
`14-27`	`apiClient.interceptors.request.use((config) => { const token = localStorage.getItem('apiforge_token'); if (token) config.headers.Authorization = `Bearer ${token}`; return config; });`	**Request Interceptor:** Automatically reads the JWT from local storage and injects the `Authorization: Bearer <token>` header into every outgoing HTTP request.	"Why use an interceptor instead of manually setting headers in every API call?" (DRY principle: eliminates boilerplate across 20+ service functions and guarantees no authenticated call forgets the token).
`30-44`	`apiClient.interceptors.response.use( (response) => response.data, (error) => Promise.reject(errorPayload) );`	**Response Interceptor:** Automatically unwraps `response.data` so components receive the payload directly, and normalizes all error structures into `{ code, message, status }`.	"How does this simplify React component code?" (Components never need `response.data.data` or ugly `err.response && err.response.data` checks).

11.2 File: `client/src/context/AuthContext.jsx` (State & Lifecycle)

> **Crucial React Pattern: Memory Leak & Unmounted State Prevention**
> In lines 12-42 of `AuthContext.jsx`: `let isMounted = true; ... return () => { isMounted = false; };`. When the application mounts, it makes an async call to `authService.getMe()` to hydrate the user profile. If the user navigates away or unmounts the component before the network request finishes, `isMounted` is set to `false`. State updates (`setUser`, `setIsLoading`) are guarded by `if (isMounted)`, preventing React memory leaks and console warnings!

11.3 File: `client/src/features/tests/TestRunsPage.jsx` (Smart Polling Engine)

Lines 62-76 implement an adaptive polling engine using `useRef` and `setInterval`:

- The page checks if any test runs are in `'queued'` or `'running'` status.
- If active tests exist, it starts a 3000ms background interval calling `fetchTests(true)`.
- Notice the argument `isPolling = true`: This suppresses the global full-page skeleton loader so the user's dashboard does not flicker while polling!
- As soon as all tests transition to terminal states, `clearInterval()` is automatically triggered to conserve network bandwidth and battery.

Section 12: Load Testing & Performance Metrics Formulations

Examiners will rigorously test your understanding of metrics, latency, and load-testing mathematics:

Metric Term	Definition & Mathematical Formula	Engineering Significance in APIForge
Virtual User (VU)	An independent, concurrent execution thread/goroutine running a continuous loop of HTTP requests simulating an active user session.	In APIForge, tests configure between 1 and 50 VUs. Unlike physical threads, k6 implements VUs as lightweight Go goroutines.
RPS (Throughput)	`Throughput = Total HTTP Requests / Total Duration (Seconds)`	Measures the transaction processing capacity of the target API before saturation occurs.
Average Latency	`Avg = (∑ Latency of all requests) / N`	**The Flaw of Averages:** Average latency hides tail spikes! If 99 requests take 10ms and 1 request takes 10,000ms, the average is ~110ms, misleading developers into believing all users have an acceptable experience.
Median Latency (P50)	The 50th percentile: 50% of requests were faster than this value, and 50% were slower.	Represents the true typical user experience, immune to extreme outlier skew.
P90 & P95 Latency	90% (or 95%) of all requests completed faster than this threshold. Only the slowest 5% exceeded it.	**The Gold Standard for SLAs:** In cloud microservices, high-percentile latency determines whether cascading microservice bottlenecks occur.
Failure Rate	`Failure Rate = (Failed Requests + Network Errors) / Total Requests`	Measures reliability. High throughput with a 20% failure rate indicates the target server is dropping connections or returning HTTP 500/503 errors.
Ramp-Up Duration	The initial period during which VUs are gradually scaled up from 0 to target VU count.	Prevents false positive spikes caused by "cold" targets (e.g. warming up database connection pools, JIT compiler optimization, and TCP connection establishment).

Section 13: Testing Strategy & Verification Framework

APIForge achieves 100% verification across 131 test cases in the monorepo:

- **Server Unit Tests (11 test files, 73 tests):** Test individual isolated units: SSRF IP parsing (IPv4, IPv6, CIDR blocks), Zod validation schemas, Mongoose models, password hashing, and token signing.
- **Server Integration Tests (5 test files, 23 tests):** Uses `supertest` to fire live HTTP requests against the Express app and an in-memory/isolated MongoDB database, asserting status codes, response headers, and database mutations.
- **Client Unit & Component Tests (9 test files, 35 tests):** Uses React Testing Library and `jsdom` to mount forms, simulate clicks, assert input validation alerts, verify protected route redirects, and check modal states.

Section 14: Top 50 Viva Examiner Questions & High-Scoring Answers

Category A: System Architecture & Design Choices (Q1–Q10)

Q1 Why did you build APIForge as a Monorepo using npm workspaces?

A monorepo with npm workspaces (`server`, `client`) allows us to maintain the frontend and backend in a unified repository with synchronized versioning, shared scripts (`npm run dev`, `npm run test`), unified CI/CD pipelines, and single-command dependency management without needing multiple git repositories.

Q2 Why did you run two HTTP servers (ports 5000 and 5001) instead of one?

Port 5000 is our management server (auth, targets, test configuration, MongoDB). Port 5001 is the isolated Demo Sandbox API. If high-concurrency load tests were directed against Port 5000, it would saturate the Node event loop, lock the database connection pool, trigger our own rate limiters, and lock users out of their dashboard. Port 5001 isolates benchmark load with zero side-effects on management operations.

Q3 What is the advantage of using React 18 + Vite over traditional Create React App (CRA)?

Vite leverages native browser ES Modules and `esbuild`, providing instantaneous server start (~200ms vs 30s in CRA) and sub-second Hot Module Replacement (HMR). Create React App is officially deprecated, relies on slow Webpack bundling, and lacks modern build optimizations.

Q4 Why choose MongoDB (NoSQL) over PostgreSQL (SQL) for APIForge?

API load tests produce polymorphic, unstructured data: custom request header maps, varying threshold rules, nested latency histograms, and status code distribution counters. In MongoDB, these are stored natively as BSON documents without requiring complex relational joins or heavy table schema migrations whenever metrics evolve.

Q5 How does Node.js handle high concurrency despite being single-threaded?

Node.js uses an event-driven, non-blocking I/O model powered by the `libuv` C library. While JavaScript execution runs on a single thread, all asynchronous I/O operations (network requests, disk access, database queries, child processes) are offloaded to OS kernel threads or the libuv thread pool. When I/O completes, callbacks are queued to the event loop.

Q6 Why did you separate `server.js` from `app.js` ?

To enable testability. Supertest requires the Express app object to bind to ephemeral ports in memory. If `app.listen(5000)` were executed inside `app.js`, integration test suites would fail with `EADDRINUSE` whenever the actual development server was running.

Q7 What is Graceful Shutdown and how does APIForge implement it?

Graceful shutdown intercepts OS termination signals (`SIGINT`, `SIGTERM`). Instead of terminating abruptly, the server stops accepting new connections, waits for active HTTP requests to finish, cleanly closes MongoDB connections via `disconnectDatabase()`, and exits. A safety-net timer with `.unref()` guarantees forced exit if requests hang past 10 seconds.

Q8 What is `.unref()` in Node.js timers?

Normally, an active `setTimeout` keeps the Node.js event loop alive until it expires. Calling `.unref()` on the timer object instructs Node that if this timer is the only thing left in the event loop, the process can exit immediately without waiting for the timer to tick down.

Q9 Why did you use React Context API instead of Redux for authentication?

Auth state in APIForge is straightforward (`user`, `token`, `isAuthenticated`, `isLoading`). React Context provides a clean, native solution with zero additional dependencies or boilerplate. Redux would introduce unnecessary actions, reducers, and store complexity for state that rarely changes outside login/logout.

Q10 Why use Tailwind CSS instead of Material UI or Bootstrap?

Tailwind CSS is utility-first and compiles away unused styles via PurgeCSS, generating tiny CSS bundles (<10KB). It avoids the heavy runtime JavaScript CSS-in-JS performance overhead of Material UI, and avoids the generic, hard-to-customize look of Bootstrap.

Category B: Security & Vulnerability Fortifications (Q11–Q20)

Q11 What is SSRF and how does APIForge prevent it?

Server-Side Request Forgery occurs when an attacker tricks a backend server into making requests to private internal resources. APIForge implements a multi-tier filter in `ssrfFilter.js`: it blocks non-HTTP schemes, rejects loopbacks (`127.0.0.0/8`), private RFC 1918 subnets (`10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`), cloud metadata (`169.254.169.254`), and IPv6 loopbacks.

Q12 What is a DNS Rebinding attack and how does your SSRF filter protect against it?

In a DNS rebinding attack, an attacker registers a public domain (e.g. `evil.com`) whose DNS A record points to `127.0.0.1` or an internal IP. A simple regex check on the URL string sees a valid domain name and allows it. APIForge calls `dns.lookup(hostname)` to resolve the real IP address before executing requests and blocks it if the resolved IP is private.

Q13 Why did you choose Bcrypt over SHA-256 for password hashing?

SHA-256 is an ultra-fast general-purpose hash function. Attackers with GPUs can calculate billions of SHA-256 hashes per second using rainbow tables. Bcrypt is an adaptive key derivation function with an adjustable work factor (salt rounds). It incorporates unique random salts and is intentionally computationally expensive (~100ms per check), making brute-force cracking mathematically infeasible.

Q14 Why does your Zod environment schema mandate a minimum 32-character JWT secret?

HMAC-SHA256 tokens signed with short, low-entropy secrets can be cracked offline in seconds using dictionary attacks. Enforcing 32 characters (256 bits of entropy) ensures the secret cannot be brute-forced even if attackers obtain signed tokens.

Q15 What does `select: false` do in the Mongoose User schema?

It excludes the `passwordHash` field from query results by default whenever `User.find()` or `User.findById()` is called. This prevents accidental exposure of password hashes in API responses. The field must be explicitly requested using `.select('+passwordHash')` only during credential verification.

Q16 What does Helmet middleware do?

Helmet sets secure HTTP response headers: `X-Content-Type-Options: nosniff` (prevents MIME-sniffing exploits), `X-Frame-Options: SAMEORIGIN` (prevents clickjacking in iframes), `Strict-Transport-Security` (forces HTTPS), and `Content-Security-Policy` (restricts unauthorized script execution).

Q17 Why is `express.json({ limit: '1mb' })` configured with a 1MB limit?

Without a size limit, an attacker could send a 100MB JSON payload in a POST request. Node.js would attempt to parse this into memory, causing event loop blocking, memory spikes, and eventual out-of-memory crashes (Denial of Service). A 1MB limit rejects oversized payloads immediately with HTTP 413 Payload Too Large.

Q18 Why is rate-limiting configured differently on auth routes vs general API routes?

General API routes allow 200 requests per 15 minutes to support active dashboard navigation. Auth routes (login/register) are strictly limited to 10 attempts per 15 minutes. This aggressively prevents credential-stuffing and brute-force password cracking attacks.

Q19 **Why do we perform an account existence check in `authenticate.js` after verifying the JWT?**

Because JWTs are stateless. If a user is deleted, suspended, or has their password changed, their JWT remains cryptographically valid until its expiration time. Checking `User.findById(decoded.id)` ensures revoked or deleted users are denied access immediately.

Q20 **Why does the error handler conceal error stacks in production mode?**

Error stack traces reveal internal file paths, operating system details, third-party library versions, and database schemas. Attackers use this information to target known vulnerabilities. In production, we return a generic message and keep detailed stacks restricted to server logs.

Category C: Backend Engineering & Express/Mongoose (Q21–Q30)

Q21 How does Express distinguish error-handling middleware from normal middleware?

By function arity (parameter count). Normal Express middleware accepts 2 or 3 arguments: `(req, res, next)`. Error-handling middleware MUST declare exactly 4 arguments: `(err, req, res, next)`. Express checks `fn.length === 4` to identify error handlers.

Q22 What does `res.headersSent` guard against in `errorHandler.js` ?

If part of the response body or headers has already been streamed to the client before an error is thrown, attempting to call `res.status(500).json(...)` will crash the Node process with `ERR_HTTP_HEADERS_SENT`. Checking `if (res.headersSent) return next(err);` delegates to Express's built-in stream termination handler safely.

Q23 What is MongoDB Error Code 11000?

It is the MongoDB Duplicate Key error, triggered when an insert or update violates a `unique: true` index constraint (such as attempting to register with an email address that already exists).

Q24 What is a Mongoose CastError and how do you handle it?

A `CastError` occurs when Mongoose fails to cast a string into the schema type (e.g. passing an invalid 5-character string where a 24-character hexadecimal MongoDB `ObjectId` is expected). The error handler intercepts this and returns HTTP 400 Bad Request instead of an internal 500 error.

Q25 Why is `autoIndex` disabled in production inside `database.js` ?

When Mongoose boots with `autoIndex: true`, it issues `createIndex` commands for all schemas. In production collections with thousands of documents, this causes significant performance degradation and collection locking. Indexes in production should be created beforehand via migration scripts.

Q26 What is the purpose of compound indexes like `{ owner: 1, createdAt: -1 }` ?

Compound indexes index multiple fields together. In APIForge, users frequently request their own test history sorted by creation time (`TestRun.find({ owner: userId }).sort({ createdAt: -1 })`). A compound index allows MongoDB to satisfy both the filter and the sort order directly from the B-tree index in O(log N) time without scanning unowned documents or performing in-memory sorting.

Q27 How does the Finite State Machine in `testRun.model.js` enforce valid transitions?

It maintains a transition map where terminal states (`completed`, `failed`, `cancelled`) have empty allowed arrays (`[]`). The `transitionTo()` method verifies that the requested transition exists in the map. Additionally, a Mongoose `pre('save')` hook checks `$locals.originalStatus` to prevent developers from bypassing the state machine via direct property assignment.

Q28 Why store an immutable `targetSnapshot` inside the TestRun document?

If a user modifies the target URL or deletes the target in the future, past test records should remain historically accurate. Embedding an immutable snapshot of `{ name, baseUrl }` ensures the test run permanently records the exact endpoint tested at that point in time.

Q29 What is an AbortController and where is it used in APIForge?

`AbortController` is a standard JavaScript interface for aborting asynchronous operations. It is used in `target.service.js` inside the target health probe: a 5-second timer calls `controller.abort()`, which cancels the fetch request if the target server hangs or fails to respond.

Q30 What is the difference between Mongoose methods and statics?

`methods` are document instance methods called on an individual instantiated record (e.g. `user.comparePassword(...)` or `testRun.transitionTo(...)`). `statics` are model-level methods called directly on the Model class itself (e.g. `User.hashPassword(...)`).

Category D: Frontend Architecture & React (Q31–Q40)

Q31 **What is an Axios Interceptor and what two interceptors are used in APIForge?**

Interceptors run code before a request is dispatched or after a response is received. APIForge uses: (1) A **Request Interceptor** that reads the token from `localStorage` and attaches the Bearer header; (2) A **Response Interceptor** that unwraps `response.data` and normalizes errors into a uniform `{ code, message, status }` structure.

Q32 **Why use `isMounted` in `AuthContext.jsx` 's `useEffect` ?**

Asynchronous operations (like fetching the user profile) may finish after a component has already unmounted (e.g. if the user navigated away). Calling `setState` on an unmounted component causes memory leaks and React warnings. The `isMounted` cleanup flag ensures state is only updated if the component is still mounted.

Q33 **How does the `ProtectedRoute` component work?**

It consumes `useAuth()`. If `isLoading` is true, it displays an animated skeleton shell. If `isAuthenticated` is false, it uses React Router's `<Navigate to="/login" state={{ from: location }} replace />` to redirect to login while preserving the intended destination in router state for post-login redirect. If authenticated, it renders child routes via `<Outlet />`.

Q34 **How does the smart polling mechanism work in `TestRunsPage.jsx` ?**

It uses a `useRef` to hold the interval ID. A `useEffect` inspects the list of test runs: if any run is `queued` or `running`, it sets a 3-second interval calling `fetchTests(true)` (passing `isPolling=true` to avoid full-screen skeleton flickering). When all tests enter terminal states, the interval is cleared immediately.

Q35 **What is the advantage of custom hooks like `useAuth()` ?**

It abstracts `useContext(AuthContext)` into a clean, reusable call. It also adds a safety assertion: if `useAuth()` is called outside an `AuthProvider`, it throws an immediate descriptive error (`"useAuth must be used within an AuthProvider"`), preventing silent runtime failures.

Q36 **Why use `useCallback` for `fetchTests` in `TestRunsPage.jsx` ?**

`useCallback` memoizes the function definition across re-renders. Because `fetchTests` is listed as a dependency in the polling `useEffect`, without `useCallback` a new function reference would be created on every render, triggering an infinite polling loop!

Q37 **What are React Router Future Flags (e.g. `v7_startTransition` , `v7_relativeSplatPath`)?**

They enable React Router v7 breaking behavioral changes early in React Router v6. This ensures smooth, zero-downtime upgrades to the next major version of the router without code breakage.

Q38 **Why store JWT in `localStorage` instead of an in-memory variable?**

Storing in an in-memory variable would log the user out on every page refresh or browser tab reload. Storing in `localStorage` provides persistent session hydration across browser reloads.

Q39 **How does `CreateTestModal.jsx` handle dynamic headers?**

It stores headers as an array of key-value objects in state: `[{ key: '', value: '' }]`. Users can add or delete rows dynamically. On submit, it converts the array into a clean JSON key-value map before dispatching to the backend API.

Q40 **What is the role of atomic UI components (Button, Input, Card, Badge, Modal)?**

Atomic design separates core UI primitives from business logic. Centralizing styles in reusable components ensures consistent dark developer theme aesthetics, standardizes accessibility attributes, and eliminates code duplication across pages.

Category E: Load Testing, k6 & Performance Metrics (Q41–Q50)

Q41 What is a Virtual User (VU) and how does it differ from a real user?

A Virtual User is an automated, continuous execution loop running requests against the target. While a real human user clicks once every few seconds and browses, a VU executes requests in rapid succession with zero think-time (unless configured), generating hundreds of times more traffic than a single human.

Q42 Why is Average Latency considered a misleading metric in load testing?

The "Flaw of Averages": Outliers get washed out. If 95% of users experience 20ms response times, but 5% experience 5000ms timeouts, the average might look acceptable (270ms), masking the fact that 1 in 20 users experiences an unacceptable outage. Percentiles (P95, P99) expose this tail latency.

Q43 What is the P95 latency percentile?

P95 is the value for which 95% of all measured request latencies were faster, and only 5% were slower. It represents the worst-case experience of typical users and is the industry benchmark for Service Level Agreements (SLAs).

Q44 What is Ramp-Up Duration and why is it important?

Ramp-up is the period where VUs are gradually scaled up (e.g. from 1 to 50 VUs over 10 seconds) rather than hitting the target with 50 VUs instantly. It allows database connection pools, thread pools, and caches to warm up, preventing artificial error spikes caused purely by cold-start socket handshakes.

Q45 What are Thresholds in load testing?

Thresholds are pass/fail criteria (SLOs) defined on metrics. For example: `http_req_duration: ['p(95)<200']` asserts that 95% of requests must complete under 200ms, and `http_req_failed: ['rate<0.01']` asserts failure rate must be under 1%. If violated, the test is flagged as failed.

Q46 Why is k6 superior to JMeter for modern load testing?

k6 is written in Go and compiles to native machine code. It uses lightweight goroutines (each taking ~2KB of memory) rather than heavy Java OS threads (which take 1MB each in JMeter). This allows k6 to generate massive traffic on modest hardware with minimal CPU/RAM consumption and no JVM garbage-collection pauses.

Q47 What is the difference between Throughput (RPS) and Bandwidth?

Throughput (Requests Per Second) measures the rate of completed application-level HTTP request/response transactions. Bandwidth (MB/s or Mbps) measures the raw volume of physical network data transferred across the wire.

Q48 What is HTTP Status 429 and what header accompanies it?

HTTP 429 indicates `Too Many Requests` (rate-limiting). It is accompanied by the `Retry-After` header, which specifies the number of seconds the client must wait before making new requests.

Q49 How does APIForge simulate load testing in Stage 6?

In Stage 6, `simulateLifecycleRun()` executes deterministic lifecycle transitions (`queued → running → completed`) with mathematically calculated metrics (RPS, P90, P95, status code counts) and clearly labels the document with `isSimulated: true`. This verifies all lifecycle state machines, progress polling, and UI charts before integrating the native k6 binary in Stage 7.

Q50 What are the 3 key takeaways of the APIForge architecture you would summarize to the examiner?

1. Resilience: Strict finite state machine lifecycle with immutable target snapshots, graceful shutdown, and per-user concurrency limits to protect system resources.
2. Security-First: Multi-layer defense-in-depth including DNS-resolving SSRF filtering, 32-char minimum entropy JWT secrets, automatic password hash exclusion, and aggressive auth rate-limiting.
3. Performance Isolation: Decoupled dual-server topology separating management operations on Port 5000 from high-throughput benchmarking on Port 5001.

End of APIForge Comprehensive Viva Voce Defense Guide • Prepared for Academic Capstone Project Evaluation • All 131 Verification Tests Passed